

Core Odoo Object Schema

MDR Devices Compliant Digital Thread — field-level design, native vs. extension, stated precisely

ARCHITECTURAL PRINCIPLE

Native Odoo Enterprise configuration carries the majority of this flow — credential gating, quarantine automation, the analytic spine that ties every financial document back to its clinical event. Five core applications (Inventory, Manufacturing, Quality, Sales, Accounting) do the heavy lifting, with **Field Service** closing the loop at the point of surgery and **CRM** carrying the physician credential record — both native, both named.

Five specific requirements exceed what standard Odoo covers out of the box and are addressed through a scoped **MedTech extension layer**, consistent with the vertical-extension pattern already proven on prior regulated device engagements: **(1)** structured UDI compliance reporting, **(2)** recall & consignment reporting depth, **(3)** multi-payer & rebate settlement — the payer-split invoicing itself (two-way *and* multi-tier) plus automated vendor-rebate match-and-trigger, **(4)** reusable-instrument sterilization-cycle tracking, and **(5)** the QA dual-authentication release gate. Each is called out explicitly below rather than folded silently into “native.”

1 `res.partner` — Credential Governance & Consignment Custody

1.1 Custom Field Registry

FIELD NAME	TYPE	DESCRIPTION
<code>x_hcp_certification_number</code>	Char	Physician/hospital license or accreditation reference
<code>x_hcp_certification_expiry</code>	Date	Drives automated block/warn logic on order confirmation
<code>x_hcp_device_qualifications</code>	Many2many → <code>product.category</code>	Implant families this HCP is credentialed to receive
<code>x_validation_state</code>	Selection: Draft / Validated / Expired / Suspended / Pending Verification	Single authoritative status consumed by Sales, Field Service, and Finance. <i>Pending Verification</i> covers a locum or visiting physician with no existing record at time of order — routes to a manual review queue rather than a hard block.
<code>x_consignee_type</code>	Selection: Hospital / Distributor / Direct	Determines consignment location routing logic

1.2 Enforcement Logic

An automated action fires on `sale.order.action_confirm()`, raising a `ValidationError` when:

- `partner_id.x_validation_state` is *Expired* or *Suspended*, OR
- `partner_id.x_hcp_certification_expiry` < today, OR
- the ordered product's category is absent from `partner_id.x_hcp_device_qualifications`.

An order against a *Pending Verification* partner routes to a manual review queue instead of a hard block — this is a five-line server action override, correctly named as a small piece of code-level logic rather than folded into a “no custom development” claim, and it is the one piece in this section that warrants a code-level (not purely Studio-config) review at each major upgrade.

1.3 Consignment Stock — On-Balance-Sheet Modeling

Direction determines the mechanism. Odoo's documented `owner_id` / "Consignment" feature is built for the *inbound* case — stock physically in your warehouse that a vendor still owns. MDR Devices' scenario is the mirror image: implants MDR Devices still owns, physically sitting at a hospital. That is the outbound/consignor direction, and a different native mechanism does the actual work.

INBOUND — WHAT `OWNER_ID` IS FOR

A vendor's stock, physically in your warehouse. Ownership tagging keeps it off your books while you hold it.

OUTBOUND — THE MDR DEVICES CASE

Your stock, physically at a hospital. Internal location type keeps it on your balance sheet until an actual sale moves it to a customer location.

Primary mechanism: internal location type. Odoo's valuation logic keys off location type, not ownership tagging: quants sitting in an internal location stay on the owning company's books regardless of physical distance from the warehouse; only a move to a customer-type location (an actual sale/delivery) removes stock from valuation. Modeling each hospital as a nested internal location (`WH/Consignment/{Hospital Name}`) is therefore the primary, correct native mechanism for keeping MDR-owned stock on MDR's balance sheet while it physically sits at the hospital. Odoo's own subcontracting-valuation documentation describes the directly analogous case — components a company still owns, physically located at a third-party site — and confirms the same mechanism.

DESIGN NOTE, STATED PLAINLY — THIS ROUTING IS DELIBERATE CONFIGURATION, NOT DEFAULT BEHAVIOUR

A confirmed sale order in standard Odoo delivers to a *customer* location and (under "invoice on delivered quantities") treats that delivery as the invoicing trigger. The consignment flow here deliberately departs from both defaults: a custom delivery route sends the SO's outbound move to the internal consignment location, and invoicing is triggered by the later *consumption* move (Value Stream, Stage 5), not the delivery. Both are achievable through route and automation configuration — but they are design decisions we make and validate explicitly in discovery, not out-of-the-box behaviour, and they are named as such here so no one discovers that distinction during UAT.

Reporting nuance to confirm in the sandbox: Odoo has a documented reporting quirk where per-location valuation breakdowns across multiple internal locations don't always reflect internal transfers correctly out of the box — aggregate company-wide valuation stays correct, but a "how much do we hold at Hospital X, by value" view may need `warehouse_id`/location made a stored, groupable field to report reliably. Confirm during discovery, separately from the underlying valuation mechanism itself.

Reverse flow — schema level. A `x_return_disposition` field on `stock.lot` (Selection: Re-Quarantine / Scrap / Return-to-Vendor / Under Investigation) is set by the receiving quality reviewer when a return delivery is processed from a consignment location, driving automated routing of the returned quant rather than leaving disposition to ad hoc warehouse judgment. A companion `x_return_reason` field creates a queryable return-reason history against the lot. Full trigger/action detail: Value Stream specification, Stage 4B.

EDGE CASE, STATED PLAINLY — THE UNMATCHED RETURN

Real hospital returns sometimes arrive with damaged labels or incomplete paperwork — no clean scan, no automatic match. For exactly this case the return process includes a manual match-and-flag queue: an unmatched return creates a `x_return_disposition = Under Investigation` placeholder record, routed to Quality for manual reconciliation against shipment history rather than being silently dropped or force-matched to the wrong lot. A named fallback, not a claim that every return resolves automatically.

Renamed from stock.production.lot in Odoo 16.

2.1 Native Foundation NATIVE — NO CUSTOM DEVELOPMENT

Odoo's Barcode app natively parses GS1 Application Identifier-formatted barcodes into structured fields (product, lot, quantity) using the existing barcode nomenclature engine — this part requires no custom development and is presented as such.

2.2 MedTech Extension Layer EXPLICITLY SCOPED

Splitting a scanned UDI further into MDR/UDI-database-reportable segments — Device Identifier, and Production Identifier broken into batch, serial, and expiry sub-fields tied directly to compliance reporting — is a targeted extension on top of that native parsing, not a native Odoo capability:

FIELD NAME	TYPE	DESCRIPTION
x_udi_di	Char	Device Identifier — inherited from product.template configuration
x_udi_pi	Char (computed)	Production Identifier — parsed from the native GS1 scan into constituent segments
x_udi_pi_batch	Char	Parsed batch/lot segment (GS1 AI 10)
x_udi_pi_serial	Char	Parsed serial segment (GS1 AI 21)
x_udi_pi_expiry	Date	Parsed expiry segment (GS1 AI 17), cross-linked to native expiration_date
x_storage_temp_min / x_storage_temp_max	Float	Cold-chain envelope for the lot, checked at each transfer against recorded storage-condition data

WHERE THE TEMPERATURE DATA COMES FROM — NAMED, NOT IMPLIED

The envelope fields above define the tolerance; they do not create the measurements. Actual temperature readings enter the system through one of two discovery-scoped integration options: data-logger import (CSV/API from the loggers already travelling with cold-chain shipments) or live sensor feeds via Odoo's IoT integration. Which one, and at what granularity (per-location daily log vs. per-shipment logger), is a named discovery decision — cold-chain evidence capture is an integration workstream, not a free by-product of two float fields.

2.3 Native Fields Leveraged NO CUSTOM BUILD REQUIRED

use_expiration_date, expiration_time, removal_time — standard Odoo Inventory “Expiration Dates” feature — govern the sterility clock without additional schema.

EDGE CASE, STATED PLAINLY — NON-CONFORMING LABELS

Hospitals and some suppliers frequently re-label incoming devices with proprietary internal tracking labels that don't conform to GS1 Application Identifier structure. These will not auto-parse. The extension layer includes a manual-entry fallback screen — DI/PI entered by hand against the same stock.lot fields, flagged x_udi_source = Manual so reporting can distinguish scanned from hand-entered records. A known, named limitation.

3.1 Master Production Schedule (MPS) NATIVE

Native Manufacturing app MPS forecasts long-lead-time raw materials against sales forecast horizons — configuration of replenishment periods per product category, no custom development. For planned surgeries, delivery scheduling keys off the confirmed surgery date on the order, so just-in-time consignment replenishment is driven by the clinical calendar rather than generic lead times.

3.2 Reordering Rules NATIVE

Min/max reordering rules scoped explicitly to the post-hold stock location (WH/Stock, not WH/Quarantine), ensuring auto-replenishment triggers off truly available-to-promise inventory rather than stock still serving its mandatory hold — this is the configuration that answers the case's “auto-replenish below the three-month quality-hold requirement” sentence directly.

THE HOLD ITSELF — COMPANY QUALITY POLICY, ENFORCED STRUCTURALLY; REGULATORY DRIVER CONFIRMED IN DISCOVERY

The three-month hold is stated in the brief as MDR Devices' own quality-hold requirement. No general MDR/GxP rule mandates a fixed 90-day quarantine, and this design does not claim one: we treat the hold as a hard company constraint, enforce it structurally, and confirm its exact regulatory or contractual driver — and whether the brief's phrasing also implies a three-month coverage reorder trigger — as a named discovery question rather than asserting a regulation that doesn't exist.

3.3 Quarantine & Quality Hold Enforcement

MECHANISM	CONFIGURATION
Routing	Receipt operation type push rule routes incoming lots to WH/Quarantine (internal location type)
Quality Check	Quality Control Point auto-generates a check on the receipt operation (visual, temperature, certificate-of-analysis review) — native Quality app
Hold Timer	Automated action validates lot.create_date + 90 days <= today
Release	Scheduled action moves the quant Quarantine → Stock only when both Quality Check = Pass and the hold timer condition are satisfied — enforced structurally, not procedurally
Failure Disposition	If Quality Check = Fail — before or after the 90-day timer elapses — the automated action routes the quant to WH/Quarantine/Disposition, blocks further movement, and opens a QA deviation activity. No path back to WH/Stock without one of three logged outcomes: Scrap, Return-to-Vendor, or Extended Hold pending investigation.

4.1 Multi-Dimensional Analytics NATIVE

Three parallel analytic plans (native multi-plan analytics, introduced in Odoo 16 when analytic plans and distributions replaced analytic tags): Cost Center, Payer Type, Product Line. Every invoice line auto-tags across all three via a distribution template keyed on product category and customer type, resolved automatically at invoice generation. This analytic spine — one shared key tying every financial document back to its originating clinical event — is the genuinely native part of the settlement story.

4.2 Payer-Split Invoicing MEDTECH EXTENSION LAYER — EXPLICITLY SCOPED, INCLUDING THE TWO-WAY CASE

Stated precisely: a sale order natively invoices a single partner. Odoo's down-payment / milestone invoicing splits amounts over time for the same customer; it does not generate invoices against two different partner_id records from one order. Splitting a single surgical consumption event across an insurer and a patient co-pay therefore sits in the named extension layer — as does the full multi-tier case (primary insurer, secondary insurer, patient balance, rebate

netting against the same event). The native analytic spine (§4.1) still carries the shared event key across every invoice the extension generates, which is what makes reconciliation structural rather than a matching exercise. Earlier drafts of this package presented the two-way split as native; that claim did not survive verification and is corrected here. Size the adjudication logic precisely during discovery once MDR Devices' real payer mix is confirmed.

4.3 Vendor Rebate Recovery EXTENSION-LAYER AUTOMATION ON NATIVE PURCHASE DATA

Purchase agreements, vendor pricelists, and the analytic spine are native and carry the raw data. The automation that makes the recovery argument real — watching cumulative purchase volumes against tiered rebate thresholds and generating the vendor credit-note claim the moment a threshold posts — is **not** an out-of-the-box Odoo engine. It is extension-layer logic within scope item (3), built on the same analytic tagging structure used on the sales side.

THE SHARPEST EBITDA ARGUMENT — RECOVERY, NOT PROJECTION

Manually tracked rebate programs routinely under-capture value MDR Devices has already contractually earned — thresholds crossed but not caught, credits owed but never claimed. Automating the match-and-trigger doesn't create new value; it stops leaving already-negotiated value uncollected. That distinction — recovery versus projection — is the one a CFO finds easiest to defend in a board meeting. The argument survives the honest mechanism statement above intact: the value case never depended on the automation being free.

5 Reusable Surgical Instrument Trays — Asset Loop (distinct from implant consumption)

Why this needs its own model. Everything in Sections 1–4 assumes a one-way consumption pattern: an implant is received, held, manufactured, shipped, and consumed once. Surgical instrument trays — named explicitly alongside implants in scope — don't follow that pattern. A tray is dispatched, used in surgery, returned dirty, sterilized, and redeployed, repeatedly, until retired. Modeling it through the same `stock.lot` consumption flow would silently treat each redeployment as a new “sale” — operationally wrong, and it would corrupt UDI/consumption reporting.

5.1 Native Foundation NATIVE

Odoo's Inventory app handles the physical moves natively: tray dispatch to `WH/Consignment/{Hospital Name}` and return to a `WH/Sterilization` internal location are ordinary internal transfers — mechanically identical to the implant moves in §1.3.

5.2 Cycle Tracking MEDTECH EXTENSION LAYER — EXPLICITLY SCOPED

Odoo has no native concept of a sterilization cycle count or reuse-limit enforcement. This is the fourth named extension-layer item:

FIELD NAME	TYPE	DESCRIPTION
<code>x_asset_type</code>	Selection: Single-Use / Reusable	Governs which lifecycle a given <code>stock.lot</code> follows
<code>x_sterilization_cycle_count</code>	Integer (computed)	Increments on each completed, passed return-to-sterilization event
<code>x_sterilization_cycle_limit</code>	Integer	Manufacturer-specified maximum reuse count before mandatory retirement
<code>x_last_sterilization_date</code>	Date	Cross-checked against hospital-reported use date before re-dispatch is permitted

5.3 Cycle Enforcement

A Quality Control Point on the `WH/Sterilization` location requires a passed autoclave/sterility check before a tray can move back to dispatch-ready stock — the same pass/block pattern proven for the inbound quality hold in §3.3. The check records the autoclave cycle reference plus cycle parameters (temperature, duration, pressure) against the `quality.check`, not just a pass/fail flag, so a disputed cycle has parameter data behind it. An automated action blocks dispatch once `x_sterilization_cycle_count >= x_sterilization_cycle_limit`, forcing retirement instead of a silent over-cycled tray reaching a hospital.

DATA-INTEGRITY BOUNDARY, STATED PLAINLY

The cycle-limit block is structural and can't be bypassed. What it can't fully prevent is a technician recording a false pass on the autoclave check itself — which is precisely why this check sits inside the same two-step release gate as quarantine release (§3.3 / GxP Validation Plan §4): a second, independently authenticated reviewer countersigns redeployment. One person's mistake alone can't put an over-cycled or improperly sterilized kit back into use.

UPGRADE PATH — PRECISE CLAIM

Studio-created fields, views, and automated actions are stored as configuration data rather than external code, which carries forward across major Odoo version upgrades far more reliably than bespoke Python modules — but “far more reliably” is the accurate claim, not “risk-free.” Odoo's own upgrade process does not guarantee zero adjustment for complex automated rules; a configuration review at each major release is the honest expectation, and it is a materially smaller task than re-validating custom code. The scoped MedTech extension fields above sit in this same lower-risk category, being Studio-based configuration rather than a standalone custom module.